

Examen de práctica 2: CNNs, RNNs, Transformers y LLMs

Aprendizaje Profundo y sus usos en Políticas Públicas · 2026

Este examen de práctica tiene el mismo formato que el examen real: tendrá 3 secciones y, al final, un formulario de arquitecturas, para que no tengas que memorizarlas. Las respuestas están al final del documento. Recuerda que tendrás 40 minutos para el examen real.

Parte A — Conceptos rápidos

Escribe V o F y justifica en una o dos líneas.

- A1. Una capa de max-pooling no tiene parámetros entrenables.
 - A2. Una convolución 3×3 con padding 1 y stride 1 conserva el alto y el ancho de la entrada.
 - A3. Si el receptive field de las unidades de la última capa convolucional cubre solo una esquina de la imagen, la red igual puede clasificar la imagen completa sin problema.
 - A4. Un modelo de lenguaje que predice uniformemente sobre un vocabulario de $|V|$ tokens tiene perplejidad $|V|$.
 - A5. BPTT truncado calcula exactamente el mismo gradiente que BPTT completo, solo que más barato.
 - A6. En una LSTM, un forget gate cercano a 0 hace que la memoria c_t conserve lo que traía.
 - A7. Si todos los pesos de atención son iguales a $1/m$, la atención se reduce a un promedio simple de los values.
 - A8. La codificación posicional aprendida generaliza mejor que la sinusoidal a secuencias más largas que las vistas en entrenamiento.
 - A9. Un modelo encoder-only tipo BERT es la elección natural para generar texto libre.
 - A10. RAG permite que un LLM responda con información actualizada sin reentrenar sus pesos.
-

Parte B — Desarrollo

B1. Aritmética de una CNN

Una red recibe imágenes en escala de grises de $32 \times 32 \times 1$ y tiene:

- Capa 1: convolución con 8 filtros de 3×3 , padding 1, stride 1, con bias.
- Capa 2: max-pooling 2×2 , stride 2.
- Capa 3: convolución con 16 filtros de 3×3 , **sin** padding, stride 1, con bias.

a) Tamaño de la salida de cada capa.

- b) Parámetros entrenables de cada capa.
- c) ¿Cuántos píxeles de la imagen original ve una unidad de la salida de la Capa 3? Muestra el cálculo.
- d) Si cambias la Capa 3 por 16 filtros de 1×1 , ¿qué mezcla y qué no mezcla esa capa? ¿Cuántos parámetros tendría?

B2. RNNs y modelos de lenguaje

- a) Una RNN simple recibe inputs de dimensión 32 y tiene estado oculto de dimensión 64. ¿Cuántos parámetros tiene la recurrencia $H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$? ¿Cambia ese número si las secuencias miden 10 o 1,000 pasos? ¿Por qué?
- b) En una GRU, ¿qué valor del update gate z_t hace que $h_t \approx h_{t-1}$? ¿Qué gate de la LSTM juega el papel análogo?
- c) El gradiente de un mini-batch tiene norma 20 y el umbral de clipping es $\theta = 5$. ¿Por qué factor se reescala? ¿Cambia su dirección? ¿Qué problema resuelve el clipping y cuál no?
- d) Un equipo compara un modelo de lenguaje sobre actas con perplejidad 40 y otro con 120, pero el segundo usa un tokenizador de caracteres. ¿Puedes concluir cuál es mejor? ¿Qué harías?

B3. Atención

Query $q = (1, 1)$; tres pares key/value con $d_k = 2$:

$$k_1 = (1, 0), v_1 = (1, 0); \quad k_2 = (0, 1), v_2 = (0, 1); \quad k_3 = (1, 1), v_3 = (1, 1).$$

Puedes usar $1/\sqrt{2} \approx 0.71$, $\sqrt{2} \approx 1.41$, $e^{0.71} \approx 2.03$, $e^{1.41} \approx 4.11$.

- a) Calcula los scores escalados, los pesos softmax y la salida.
- b) ¿Qué pasaría con los pesos si los scores fueran 10 veces más grandes? ¿Qué tiene que ver esto con dividir entre $\sqrt{d_k}$?
- c) ¿Qué es la máscara causal y qué le pasaría al entrenamiento de un decoder si la quitas?
- d) ¿Por qué el bloque Transformer tiene conexiones residuales y LayerNorm alrededor de cada subcapa?

Parte C — Diseño aplicado

Una secretaría de salud quiere predecir la **demanda semanal de consultas** en cada uno de sus 120 centros para asignar personal. Tiene 4 años de historia por centro (unos 200 puntos por centro) y variables como semana del año, campañas activas y clima. Un equipo propone:

Entrenar una LSTM **bidireccional** sobre las series, mezclando al azar todas las semanas de todos los centros en train (80 %) y validación (20 %), y reportar el error promedio.

- a) Señala dos problemas de la propuesta: uno de arquitectura y uno de protocolo de validación. Explica qué saldría mal y qué harías en su lugar.
- b) Además de la LSTM, ¿qué línea base simple exigirías antes de creer en el modelo?

- c) La secretaría también quiere clasificar automáticamente 5,000 quejas escritas ya etiquetadas en 8 categorías, y luego clasificar las 40,000 que llegan al año. Un agente propone “un LLM con prompting zero-shot”. ¿Prompting, RAG o fine-tuning? Justifica y di cuándo cambiarías la decisión.
- d) ¿Qué métrica adicional al error promedio pedirías antes de usar el modelo de demanda para asignar personal, y qué modo de falla te preocupa más?
-

Formulario: arquitecturas

Convolución. salida = $\left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$ por dimensión; #parámetros = $c_o c_i k_h k_w + c_o$.

Receptive field. $r_l = r_{l-1} + (k_l - 1) j_{l-1}$, $j_l = j_{l-1} s_l$, con $r_0 = j_0 = 1$.

RNN simple. $H_t = \phi(X_t W_{xh} + H_{t-1} W_{hh} + b_h)$, $O_t = H_t W_{hq} + b_q$.

LSTM. $f_t, i_t, o_t = \sigma(\cdot)$; $\tilde{c}_t = \tanh(\cdot)$; $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$; $h_t = o_t \odot \tanh(c_t)$.

GRU. $r_t, z_t = \sigma(\cdot)$; $\tilde{h}_t = \tanh(x_t W_{xh} + (r_t \odot h_{t-1}) W_{hh} + b_h)$; $h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$.

Gradient clipping. $g \leftarrow \min\left(1, \frac{\theta}{\|g\|}\right) g$.

Perplejidad. $\exp\left(-\frac{1}{n} \sum_t \log P(x_t | x_{<t})\right)$.

Atención. $a(q, k_i) = \frac{q^T k_i}{\sqrt{d_k}}$, $\alpha_i = \frac{e^{a_i}}{\sum_j e^{a_j}}$, salida = $\sum_i \alpha_i v_i$; máscara $A = \text{softmax}(S + M)$, $M_{ij} \in \{0, -\infty\}$.

Multi-head. $h_i = \text{Attention}(qW_i^{(q)}, KW_i^{(k)}, VW_i^{(v)})$, $[h_1; \dots; h_H] W^o$.

Codificación posicional. $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$, $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$.

Bloque residual. $h_{k+1} = h_k + F(h_k)$.

Respuestas orientativas

Parte A

- A1. **V.** Pooling solo toma el máximo (o promedio) de una ventana; no hay pesos que aprender. Sí tiene hiperparámetros: ventana, stride, padding.
- A2. **V.** $(n + 2 \cdot 1 - 3)/1 + 1 = n$. Es el “same padding” para kernels impares: $p = (k - 1)/2$.
- A3. **F.** Una unidad solo puede usar la información de su receptive field; si no cubre la imagen, la decisión ignora el resto. Por eso se apilan capas, se usa stride/pooling o agregación global antes de clasificar.
- A4. **V.** Cada token tiene probabilidad $1/|V|$; la cross-entropy es $\log |V|$ y $\exp(\log |V|) = |V|$: el modelo “duda” entre $|V|$ opciones.
- A5. **F.** Truncar a τ pasos ignora las dependencias más largas que τ : es una aproximación del gradiente, más barata y estable, pero limita lo que la red puede aprender del pasado lejano.
- A6. **F.** Al revés: $c_t = f_t \odot c_{t-1} + \dots$; con $f_t \approx 0$ se **borra** la memoria anterior. Para conservarla se necesita $f_t \approx 1$.
- A7. **V.** $\sum_i \frac{1}{m} v_i$ es el promedio de los valores: average pooling. La atención generaliza esto con pesos que dependen de la query.
- A8. **F.** La aprendida solo tiene vectores para las posiciones vistas en entrenamiento; la sinusoidal se define para cualquier posición y por eso extrapola mejor (aunque no perfectamente).
- A9. **F.** BERT se entrena con MLM viendo contexto de ambos lados y no está diseñado para generar de izquierda a derecha; para generación libre se usan decoder-only (GPT) con máscara causal, o encoder-decoder para seq2seq.
- A10. **V.** RAG recupera documentos relevantes y los pone en el prompt; el conocimiento vive en el corpus, que se puede actualizar, y la respuesta puede citar la fuente. Los pesos no cambian.

Parte B

- B1.** a) Capa 1: $32 \times 32 \times 8$. Capa 2: $16 \times 16 \times 8$. Capa 3: $\lfloor (16 - 3)/1 \rfloor + 1 = 14$: $14 \times 14 \times 16$. b) Capa 1: $8 \cdot 1 \cdot 9 + 8 = 80$. Capa 2: 0. Capa 3: $16 \cdot 8 \cdot 9 + 16 = 1,168$. c) Conv 1: $r = 3, j = 1$. Pooling: $r = 3 + (2 - 1) \cdot 1 = 4, j = 2$. Conv 3: $r = 4 + (3 - 1) \cdot 2 = 8$. Una unidad ve 8×8 píxeles. d) Una 1×1 mezcla canales en cada posición pero no píxeles vecinos: es una capa lineal $\mathbb{R}^8 \rightarrow \mathbb{R}^{16}$ compartida en todas las posiciones. Parámetros: $16 \cdot 8 \cdot 1 \cdot 1 + 16 = 144$.
- B2.** a) W_{xh} : $32 \cdot 64 = 2,048$; W_{hh} : $64 \cdot 64 = 4,096$; b_h : 64. Total 6,208. No cambia con la longitud: los mismos parámetros se reutilizan en cada paso; eso es lo que distingue una RNN de una red “desenrollada” con pesos distintos por paso. b) $z_t \approx 1$ da $h_t \approx h_{t-1}$ (conserva). El análogo en LSTM es el forget gate $f_t \approx 1$ (con $i_t \approx 0$). c) Factor $\theta/\|g\| = 5/20 = 0.25$; la dirección no cambia, solo la norma. Resuelve los gradientes que **explotan**; no ayuda con los que se desvanecen (un gradiente pequeño no se toca). d) No. La perplejidad es por token; un tokenizador de caracteres hace muchas más predicciones, cada una sobre un vocabulario chico, y su número no está en la misma escala. Comparar con el mismo tokenizador, o convertir ambos a bits por carácter o a la log-verosimilitud total del mismo texto.
- B3.** a) Scores: $q^T k_i / \sqrt{2} = (0.71, 0.71, 1.41)$. Exponenciales $(2.03, 2.03, 4.11)$, suma 8.17; $\alpha \approx (0.25, 0.25, 0.50)$. Salida: $0.25(1, 0) + 0.25(0, 1) + 0.50(1, 1) = (0.75, 0.75)$. b) Con scores 10 veces mayores, $(7.1, 7.1, 14.1)$: el softmax se satura, $\alpha \approx (0, 0, 1)$, y el gradiente respecto a los scores es casi cero. Dividir entre $\sqrt{d_k}$ evita que los scores crezcan con la dimensión (la varianza de $q^T k$ crece

como d_k). c) La máscara causal pone $-\infty$ en los scores hacia posiciones futuras, así que cada token solo atiende al pasado. Sin ella, el token t ve x_{t+1} , que es su objetivo: la pérdida de entrenamiento se desploma por fuga, pero en generación el futuro no existe y el modelo no sirve. d) Los residuales dan un camino directo para la señal y el gradiente ($h + \text{Sublayer}(h)$), lo que permite apilar muchas capas sin que el gradiente se desvanezca; LayerNorm mantiene la escala de las activaciones estable en cada posición. Es la misma idea que ResNet.

Parte C

- a) *Arquitectura*: la bidireccional usa el futuro de la serie para representar cada semana; para predecir demanda hacia adelante ese futuro no existe. Usar LSTM/GRU unidireccional o una ventana fija. *Validación*: mezclar semanas al azar mete el futuro de cada centro en el entrenamiento y semanas vecinas (casi iguales) en ambos lados: el error es optimista. Partir **por tiempo** (entrenar hasta cierta fecha, validar después) y reportar también por centro.
- b) Líneas base: “misma semana del año anterior”, promedio móvil o un modelo lineal con las mismas variables. Si la LSTM no las supera en el split temporal, no vale su complejidad.
- c) Con 5,000 quejas etiquetadas, 8 categorías fijas y 40,000 al año, **fine-tuning** (LoRA sobre un encoder o un LLM pequeño) da etiquetas consistentes y baratas por documento, y se evalúa con F1 por clase. Prompting zero-shot sirve como línea base rápida pero desperdicia las 5,000 etiquetas; RAG no es para clasificar. Cambiaría si las categorías cambian con frecuencia o si hubiera muy pocas etiquetas (few-shot), o si la tarea fuera responder preguntas citando quejas (RAG).
- d) Error **por centro** y por tipo de centro (rural/urbano, tamaño), no solo el promedio; y desempeño en semanas atípicas (campañas, brotes). Modo de falla: subestimar la demanda justo en los centros pequeños o rurales, que son los que menos historia tienen y donde el error se traduce en falta de personal.